

API GATEWAY — SYSTEM DESIGN & INTERVIEW REFERENCE V1

One entry point for API traffic — what belongs in it, what must never, and how it fails
control plane vs data plane · routing · auth · rate limiting · resilience · observability · anti-patterns

1 · WHAT AN API GATEWAY IS — AND WHAT IT IS CONSTANTLY CONFUSED WITH

start here — the definitions, the neighbouring components, and the one rule that keeps a gateway healthy

The definition

An API gateway is a **reverse proxy that owns the cross-cutting concerns of API traffic** — authentication, rate limiting, routing, observability — so that each service behind it does not have to implement them separately.

The problem it solves, stated as **arithmetic**. With 40 services, doing auth, rate limiting, request logging, CORS and TLS in each one means 40 implementations, 40 sets of bugs, and 40 places to change when a policy changes. Contrasting them makes it one — and that consistency, more than any single feature, is the reason gateways exist.

The one rule

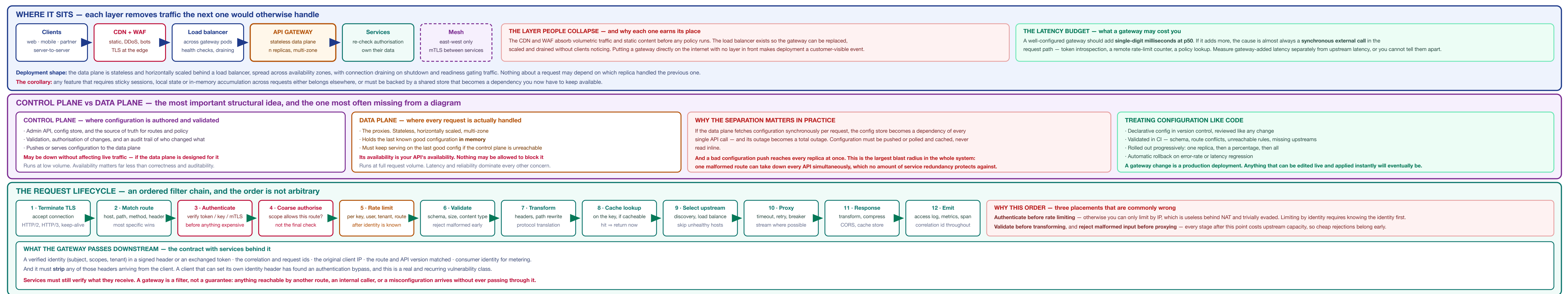
The **gateway handles concerns that are the same for every service. Nothing else**. The moment it contains logic that is true for only one service, it has become a shared component that every team must change and no team owns — and it will be the slowest part of your delivery process within a year.

North-south, not east-west

- **North-south** — traffic entering the system from outside. This is the gateway's job
- **East-west** — service-to-service traffic inside the boundary. This is a **service mesh's** job
- Routing internal calls through the gateway adds a hop, a bottleneck and a failure domain for no benefit

2 · REFERENCE ARCHITECTURE — CONTROL PLANE, DATA PLANE, AND THE REQUEST LIFECYCLE

the separation that determines whether a configuration change is routine or an outage



3 · ROUTING

matching, rewriting, shifting

What a route can match on

Host: api.example.com vs partner.example.com — often the first split

Path: Prefix, exact, or regex. Prefer prefix and exact; regex routes are slow to evaluate and hard to reason about

Method: Lets read and write traffic take different policies

Header: API version, tenant, client type, feature flag

Weight: Percentage splits for canary and blue/green

Precedence must be explicit. When two routes could match, the winner should be decided by a documented rule — usually most-specific-wins — not by declaration order in a file that several teams edit. Ambiguous precedence produces routing that changes when an unrelated team adds a rule.

Rewriting, carefully

An external contact stays stable.

api/orders/{id} → order-service:8080/orders/{id}

An unstable internal alias changes freely.

api/orders/{id} → order-service-v2:8080/api/orders/{id}

This indirection is one of the gateway's most valuable properties. Services can be renamed, split, merged or reimplemented without any client changing a URL — which is what makes a stranger migration practical.

Kubernetes: Ingress and Gateway API

- Ingress — the original resource. Simple, and its limitations pushed almost every capability into vendor-specific annotations
- Gateway API — the successor, with a **route-oriented** split: cluster admins own the GatewayClass, platform teams own Gateway, application teams own HTTPRoute.
- GatewayClass, Gateway and HTTPRoute are GA; some route types remain experimental
- They coexist. Most controllers support both, so migration is incremental — new services on HTTPRoute, existing Ingress untouched

The **route separation is the real advance**. It is what lets application teams self-serve routes without being able to change TLS, listeners or which routes are permitted — the exact bottleneck that makes a shared gateway painful.

5 · RATE LIMITING, QUOTAS AND LOAD SHEDDING

the algorithms, what each actually guarantees, and the distributed-counter decision nobody documents

The algorithms

Algorithm: Behaviour · trade-off

Fixed window: Count per calendar window. Trivial and memory-light. Allows a double burst at the boundary — the tail of one window plus the head of the next

Sliding window log: Store a timestamp per request. Exact, and the memory cost scales with traffic

Sliding window counter: Weighted blend of the previous and current window. Approximate, cheap, no boundary burst. The usual practical choice

Token bucket: Tokens refill at a steady rate; a request costs one. Permits bursts up to the bucket size — usually what you want for a public API

Leaky bucket: Requests drain at a fixed rate. Smooths output and protects a fragile upstream, at the cost of queuing latency

Choose by what you are protecting. Token bucket when clients legitimately burst and the upstream can absorb it. Leaky bucket when the upstream cannot — a legacy system with a fixed connection pool, for instance. Sliding window counter when you simply want a fair, cheap, general-purpose limit.

Dimensions to limit on

- Per API key or client — the commercial dimension, and what quotas are billed against
- Per user — fairness within one client application
- Per tenant — stops one customer starving the others
- Per route — an expensive endpoint deserves a tighter limit than a cheap one
- Per IP — only useful before authentication, and weak behind NAT or a proxy

Rate limit and quotas are different things. A rate limit is short-term protection — requests per second. A quota is a commercial allowance — calls per month. They are enforced differently, breached differently, and should return different errors.

7 · RESILIENCY

and how it backfires

Control: Purpose · setting

Timeout: Per route, not one global value. A report endpoint and a health check have nothing in common

Retry: Idempotent methods only, with backoff and jitter. Two attempts, not five

Circuit breaker: Stop calling an upstream that is already failing; fail fast instead

Outlier detection: Expect a single unhealthy instance from the pool rather than failing the whole route

Bulkhead: Cap concurrency per upstream so one cannot exhaust the gateway's capacity

Load shedding: Reject early under overload, preserving capacity for what matters most

Retry amplification is the classic gateway-caused outage. The client retries three times, the gateway retries three times, and the service mesh retries three times — 27 requests arrive at a service that is already struggling, and the retries themselves keep it down. Decide which single layer owns retry, and disable it explicitly everywhere else.

A retry budget is better than a retry count. Cap retries at a small percentage of total traffic, so retries cannot exceed a fraction of load no matter how many requests are failing. This is what stops a partial outage becoming a full one.

Health checking

- Active — the gateway probes upstreams on a schedule
- Passive — it observes real responses and rejects those that fail. Cheaper and faster to react
- Do not fail a readiness check because a downstream is down, or one dependency's outage removes every gateway replica simultaneously

Degrade rather than fail where the domain allows. A stale cached response, an empty list, or a partial payload is usually better than a 502 — but this must be an explicit per-route decision, not an accident of configuration.

10 · OPERATING IT — OWNERSHIP, BLAST RADIUS AND THE ORGANISATIONAL FAILURE

the part that decides whether a gateway accelerates delivery or becomes the thing everyone waits on

The organisational failure mode

A shared gateway with a central owning team becomes a bottleneck, and it happens gradually enough that nobody notices until it is severe. Every new route, every limit change and every header tweak becomes a ticket to one team, who are also on call for it. Delivery for forty teams throttles to the throughput of one.

What fixes it

- Self-service within guard rails. Teams own their own routes; the platform team owns the listeners, TLS, defaults and what is permitted
- Policy as code, reviewed in the owning team's repository — not a form request into a queue
- Validation in CI that catches conflicts, unreachable rules and missing upstreams before merge
- Sensible defaults so the common case needs no decision — a route with no explicit timeout still gets one

This is precisely what the **route-oriented split** in Kubernetes Gateway API encodes — cluster admin, platform team and application team as distinct resources with distinct owners. The ideas work outside Kubernetes too, and is worth adopting whatever the product.

11 · ANTI-PATTERNS AND PRODUCTION FAILURES

how a gateway goes from useful to load-bearing in the worst sense

Design anti-patterns

Anti-pattern: Why it hurts

Business logic in the gateway: The one to refuse first. Logic that belongs to a service ends up owned by no team, tested by nobody, and deployed on someone else's schedule

Heavy aggregation: One request funneling out to eight services makes the gateway a distributed monolith with the worst availability in the system. If a client needs a compound view, that is a BFF owned by the client team

Safe authorisation: Any path that bypasses the gateway — an internal caller, a second ingress, a misconfiguration — is unauthenticated access

Doing the mesh's job: Routing east-west traffic through the gateway adds a hop and a shared failure domain for no benefit

Statful gateway: Session affinity or local accumulation prevents free scaling and restarts, which is the property that made it safe

Live editing: An admin UI that applies changes instantly guarantees an unreviewed change reaches production

Production failures

Failure: What you see & the fix

Bad config push: Every API fails at once. Validate in CI, roll out progressively, not back automatically

JWKS fetch failure: All authentication fails after a key rotation. Cache keys, refresh proactively, tolerate a stale cache briefly

Rate-limit store down: Either unlimited traffic or none. Decide fail-open versus fail-closed in advance, with a local fallback

Retry amplification: A struggling service is kept down by retries. Retry budget, and one owning layer

Connection exhaustion: New requests refused while CPU is idle. Raise limits, bound pools per upstream, tune keep-alive

Cardinality explosion: Metrics cost and query time both spike. Label with route templates, never raw paths

Cache key omission: One user receives another's data. Include identity in the key; test with two accounts

The questions to be ready for

- **Is the gateway a single point of failure?** Structurally yes, which is why it is stateless, multi-replica, multi-zone, fronted by a load balancer, and holds last-known-good config so a control-plane outage does not stop traffic
- **Why not put path in every service?** You still do — the gateway does coarse validation for consistency and early rejection; services do fine-grained authorisation because only they hold the data to decide
- **Gateway or service mesh?** Different traffic. North-south versus east-west. Large systems run both
- **Would you build one?** Almost never. The proxy is the easy part; the control plane, hot reload, observability and edge cases are years of work that mature products have already done
- **What would you remove?** Aggregation, any business logic, and any synchronous external call in the filter chain

The summary worth ending on: A gateway is valuable exactly in proportion to how boring it is. Uniform policy, no per-service special cases, no logic that needs testing, and a configuration that teams change themselves. The moment it becomes interesting, it has become a distributed monolith at the front door of the system.

12 · PRODUCT CHOICES — ADVANTAGES, DISADVANTAGES AND WHEN

the differences that matter are the control plane and the operating model, not the feature list

Product: Character: Advantages

NGINX: Reverse proxy with gateway capability added: Extremely fast and well understood; tiny footprint; ubiquitous operational knowledge

Envoy: Modern proxy with a dynamic configuration API: Hot reload without dropping connections; outstanding observability; xDS makes it programmable; the substrate under many other products

Kong: Envoy-era gateway with a plugin ecosystem: Rich plugins, declarative config, good Kubernetes integration, strong API-management features

Traefik: Configuration discovered from the platform: Automatic service discovery, automatic certificates, very quick to adopt

Managed cloud gateway: Fully hosted, provider-integrated: Nothing to operate, scales fast, integrates with the provider's identity, logging and functions

Full API management (Apigee, Tyk and similar): Gateway plus portal, monetisation and lifecycle: Developer portal, key and plan management, analytics, monetisation — the business side of an API programme

Gateway API implementation (any Kubernetes-native, controller): Kubernetes-native, route-oriented: Standard resources across vendors; clean separation of admin, platform and application ownership; portable

The **honest position on building your own**. Writing a proxy that forwards a request is a weekend. Writing hot configuration reload, graceful draining, connection pooling, retry budgets, distributed rate limiting, JWKS rotation, TLS management, per-route observability and the operational tooling around all of it is several years — and every one of those is a place to get it subtly wrong in production. Adopt a product, and spend the effort on the configuration and the ownership model instead, which is where the outcomes actually differ.

13 · READINESS CHECKLIST

before a gateway sits in front of production traffic

□ Data plane is stateless, multi-replica and spread across zones

□ Serves last-known-good config when the control plane is unreachable

□ No synchronous config read in the request path

□ Configuration is version-controlled and reviewed like code

□ Config validated in CI — schema, route conflicts, unreachable rules, missing upstreams

□ Config rollout is progressive, with automatic rollback

□ Gateway upgrades canaried, never applied in place to all replicas

□ Route precedence documented and deterministic

□ Timeout set per route, derived from measurement

□ Retries only on idempotent methods, with backoff and jitter

□ One layer owns retry; a retry budget caps amplification

□ Circuit breaker and outlier detection configured per upstream

□ Connection pools bounded per upstream

□ Readiness does not fail because a downstream is down

□ Connection draining on shutdown, verified under load

□ JWT algorithms pinned; a lg header never trusted

□ Issuer and audience verified, not only the signature

□ JWKS cached with rotation by k.i.d. and refresh rate limited

□ Client-supplied identity headers stripped before forwarding

□ Forwarded headers built from verified content, allow-listed not deny-listed

□ Services re-authorise — the gateway is not the only check

□ Cross-tenant access tests exists and passes

□ Rate limits per key, user, tenant and route

□ Fail-open or fail-closed decided to the limiter store, and documented

□ Rate-limit headers returned on success as well as on 429

□ Quotas distinguished from rate limits, with different errors

□ Load shedding configured independently of rate limiting

□ Cache keys include every varying dimension, identity included

□ Cache tested with two accounts for cross-user leakage

□ Metrics labelled with route templates, never raw paths

□ Correlation id accepted, generated, propagated and returned

□ Gateway-added latency measured separately from upstream

□ Alerts on 5xx, 4xx spikes, 401/403 spikes, 429 rate and config skewed

□ Request schema validated at the edge against the published contract

□ Deprecation and sunset headers emitted for retiring versions

□ Per-consumer, per-version usage tracked so retirement is possible

□ Teams self-serve routes within platform-owned guard rails

□ Public, partner and internal traffic isolated to listener and policy

□ Load tested independently of the services behind it

□ No business logic, no heavy aggregation, no per-service special cases

The one that summarises the **sheet**: take the gateway out of the path for one route and ask what a client could now do that they could not before. If the answer is "reach the service without authentication", the security model depends on a proxy rather than on the services — and every internal caller, second ingress and future misconfiguration is already an exposure.